

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA152 Microcontrollers

Last Name: Martin
First Name: Ariane

Date: September 24, 2026
Subject/Section: BCA152 - B186.1

Laboratory Report
Real-Time Multisensor Room Monitoring System

1. Problem and Requirements

The laboratory problem was to design a simulated ESP32 embedded system that demonstrates real FreeRTOS concurrency rather than placing all behavior inside one continuous loop. The system had to monitor temperature, humidity, ambient light, and motion; provide an OLED user interface controlled by a rotary encoder; activate an alarm for abnormal temperature; and change between ACTIVE and INACTIVE operating states based on motion activity.

The implementation uses ESP-IDF with native FreeRTOS APIs in PlatformIO. Arduino-specific APIs were not used. Wokwi was used as the hardware simulation environment, allowing the DHT22 temperature and humidity values, photoresistor illumination, PIR motion, rotary encoder, OLED, and buzzer to be exercised without physical hardware.

The required real-time concepts were implemented with five meaningful tasks, explicit priorities, blocking behavior, `vTaskDelayUntil()` for periodic execution, queues, a mutex, an event group, a direct task notification, a state machine, inter-task communication, and shared-resource protection. Each RTOS object was assigned a functional role rather than being declared only to satisfy a checklist.

2. System Architecture and Design

The system is divided into hardware-facing modules and FreeRTOS tasks. The ESP32 receives temperature and humidity from the DHT22 on GPIO 15, ambient-light data from the LDR analog output on GPIO 34 (ADC1 channel 6), motion from the PIR sensor on GPIO 27, and user input from the rotary encoder on GPIOs 32, 33, and 25. The SSD1306 OLED uses I2C on GPIO 21 (SDA) and GPIO 22 (SCL), while the buzzer is controlled through GPIO 26.

At the software level, `app_main()` performs system initialization, creates the RTOS communication and synchronization objects, initializes the modules, and creates the application tasks. Sensor acquisition, display ownership, input handling, motion/state behavior, and alarm behavior are separated so that each task has one clear responsibility.

The operating-state machine has two states: ACTIVE and INACTIVE. The system begins ACTIVE. If no motion is detected for approximately 15 seconds, MotionTask changes the system to INACTIVE and the OLED is powered off. Motion detected while INACTIVE immediately returns the system to ACTIVE.

The ACTIVE condition is represented through EVENT_ACTIVE in the FreeRTOS event group so other tasks can observe the state without relying only on an unsynchronized global flag.

3. FreeRTOS Architecture

The task priorities were selected according to scheduling urgency, not general importance. InputTask and MotionTask use priority 3 because they respond to external events where quick reaction is desirable. SensorTask and AlarmTask use priority 2 because short scheduling delays are acceptable for environmental sampling and alarm evaluation. DisplayTask uses priority 1 because updating the OLED is less time-critical than input, motion, or sensor processing.

SensorTask uses vTaskDelayUntil() to maintain an approximately fixed two-second sampling period and reduce accumulated timing drift. InputTask and AlarmTask spend most of their time blocked waiting for notifications or queue data. DisplayTask blocks on sensor_queue. MotionTask performs a short check and then calls vTaskDelay(), preventing a continuous busy loop. This blocking behavior allows the scheduler to run other Ready tasks and reduces unnecessary CPU consumption.

Two SensorData queues are used because FreeRTOS queue receive operations remove messages. If DisplayTask and AlarmTask consumed one shared queue, they would compete for messages instead of both receiving every sensor update. SensorTask therefore sends a copy to sensor_queue for DisplayTask and alarm_queue for AlarmTask.

The event group contains EVENT_ACTIVE, EVENT_MOTION, and EVENT_ALARM bits. A direct task notification is used as a lightweight event signal from the rotary-encoder GPIO interrupt to InputTask. A mutex named serialMutex protects runtime serial output so multiple tasks do not intentionally write multi-line diagnostic output at the same time.

Table 1. FreeRTOS Task Design

Task	Responsibility	Trigger / Period	Priority	IPC / Sync	Typical Blocked Condition
SensorTask	DHT22 + LDR measurements	Every 2 s	2	sensor_queue + alarm_queue	vTaskDelayUntil()
DisplayTask	Own/update SSD1306 OLED	Sensor update	1	sensor_queue	Waiting for queue data
InputTask	Rotary encoder navigation	Encoder interrupt/event	3	Direct task notification	Waiting for notification
MotionTask	PIR + ACTIVE/INACTIVE state	100 ms check / event	3	Event group	vTaskDelay()
AlarmTask	Temperature alarm + buzzer	Sensor update	2	alarm_queue + event_group	Waiting for queue data

4. Implementation

Sensor acquisition was implemented as a dedicated module. The DHT22 is read through a custom ESP-IDF-compatible implementation, while the LDR uses the ESP32 ADC. The relative light percentage is calculated so that greater simulated illumination produces a greater displayed percentage. SensorData contains temperature, humidity, lightLevel, and motionDetected fields and is copied through the queues.

DisplayTask is the single owner of the OLED. This avoids several tasks writing to the same I2C display concurrently. The rotary encoder changes a display-mode value among Temperature, Humidity, Light, and Motion pages. Encoder GPIO activity wakes InputTask through a direct task notification instead of continuous polling.

AlarmTask evaluates the temperature received from alarm_queue. Temperatures below 18 C produce a low-temperature alarm, temperatures above 30 C produce a high-temperature alarm, and values from 18 C through 30 C are normal. The buzzer and EVENT_ALARM bit follow this decision.

The implementation deliberately separates hardware-independent decision logic from hardware-dependent code. Temperature-alarm evaluation, display-mode navigation, and system-state evaluation can therefore be compiled in the native test environment and verified without running the ESP32 simulator.

5. Verification and Testing

Verification combined native unit tests, Wokwi functional testing, and deliberate fault experiments. The final native test run reported 14 test cases with 14 succeeded. These tests verify decision logic that can be separated from hardware: alarm boundaries, OLED page navigation including wrap-around, and ACTIVE/INACTIVE state transitions.

Wokwi functional verification exercised the complete integrated system. Sensor values were changed interactively, the encoder was rotated in both directions, abnormal and normal temperatures were tested, and PIR/no-motion behavior was observed. A final screen recording was also added to the repository as supplementary functional evidence.

Functional testing exposed an implementation issue in the original LDR percentage mapping: the displayed percentage moved in the opposite semantic direction. The calculation was corrected so greater simulated illumination produces a greater relative light percentage. This is an example of verification finding a behavior problem that compilation alone could not detect.

Table 2. Native Unit-Test Summary

Group	Tests	Result
Temperature alarm logic	17 C low; 18 C normal; 25 C normal; 30 C normal; 31 C high	5/5 passed
Display navigation	Next/previous transitions and wrap-around behavior	5/5 passed
System-state logic	ACTIVE hold; timeout to INACTIVE; motion wake; INACTIVE hold	4/4 passed
TOTAL	14 meaningful unit tests	14/14 passed

Table 3. Wokwi Functional Verification Record

ID	Stimulus	Expected Result	Actual Observation	Result
FT-01	Change DHT22 temperature	Displayed temperature updates	Temperature changed in terminal/OLED	PASS

FT-02	Change DHT22 humidity	Displayed humidity updates	Humidity changed in terminal/OLED	PASS
FT-03	Change light level	Relative light value changes	Light percentage changed; mapping was corrected during verification	PASS
FT-04	Rotate encoder clockwise	Next OLED page	CW event changed to next page	PASS
FT-05	Rotate encoder counterclockwise	Previous OLED page	CCW event changed to previous page	PASS
FT-06	Set temperature > 30 C	Alarm activates	High-temperature message and buzzer activation observed	PASS
FT-07	Return temperature to normal	Alarm stops	Normal state restored and buzzer stopped	PASS
FT-08	Generate PIR motion	System ACTIVE	Motion maintained/returned ACTIVE state	PASS
FT-09	No motion for ~15 s	System INACTIVE	System entered INACTIVE; OLED turned off	PASS
FT-10	Motion while INACTIVE	System returns ACTIVE	System woke and OLED turned on	PASS

Table 4. Deliberate Fault Experiments

Experiment	Deliberate Change	Observation	Engineering Interpretation
1 - Remove Blocking	Temporarily removed MotionTask vTaskDelay(100 ms)	Terminal became delayed/froze at times; encoder response was delayed; Task Watchdog identified MotionTask on CPU 0	A high-priority task that never blocks can consume excessive CPU time and reduce scheduling opportunities. Delay was restored.
2 - Change Priority	Temporarily changed SensorTask priority from 2 to 5	No obvious starvation; sensor updates and encoder remained responsive	SensorTask performs bounded work then blocks with vTaskDelayUntil(), so lower-priority tasks still run. Priority 5 was unnecessary and was restored to 2.
3 - Remove Mutex	Temporarily removed mutex protection around SensorTask multi-line serial output	No obvious character/line interleaving occurred in that run	One successful run does not prove thread safety. Without the mutex, mutual exclusion is not guaranteed. Mutex protection was restored.

6. Static Code Analysis

PlatformIO Check with cppcheck was executed on the final ESP32 environment. The final summary reported 0 HIGH, 0 MEDIUM, and 7 LOW findings. The cppcheck environment status was shown as FAILED because the tool still reported the seven low-severity unusedFunction findings. The report does not treat this as a clean static-analysis pass; the findings were inspected and interpreted individually.

An earlier run contained eight low-severity findings. One actionable style issue was found in oled.c: an array that was never modified had not been declared const. It was corrected, reducing the count to seven. The remaining findings are unusedFunction reports produced in the analysis context even though the functions are reached through ESP-IDF framework behavior, C/C++ module interaction, or the native test path.

Table 5. Static Analysis Findings and Actions

Finding / Tool Output	Severity	Interpretation / Action
oled.c array not modified but not const (initial run)	Low style	Actionable; changed array declaration to const.
evaluateTemperature reported unused	Low style	Retained; used through C/C++ application/test paths.
dht22_read reported unused	Low style	Retained; used by sensor module.
app_main reported unused	Low style	Retained; ESP-IDF framework entry point.
oled_init / oled_write_string / oled_set_power reported unused	Low style	Retained; called from C++ display module.
evaluateSystemState reported unused	Low style	Retained; used by state logic/test path.

7. Engineering Discussion

A major design trade-off was using separate queues for DisplayTask and AlarmTask. This uses slightly more RAM than a single queue, but it guarantees that both consumers receive each SensorData update. Using one queue with two consumers would incorrectly distribute messages between them.

The project uses event-driven blocking wherever practical. The encoder uses an interrupt plus direct task notification, while DisplayTask and AlarmTask block on queues. This is preferable to continuous polling because it reduces CPU usage and makes task states easier to reason about.

Several problems were encountered during development. The LDR percentage was initially reversed and was corrected after functional verification. Removing MotionTask blocking during a deliberate fault experiment triggered the ESP32 Task Watchdog and reduced responsiveness, demonstrating why high-priority tasks must block or delay. Removing the serial mutex did not visibly corrupt output in one run, but it removed the guarantee of mutual exclusion, so the mutex was restored.

Static analysis also required interpretation rather than blindly deleting reported functions. cppcheck reported framework entry points and cross-language functions as unused. Only the clearly actionable const issue was changed. The final firmware build succeeded and all 14 unit tests passed.

The final build still prints a flash-size configuration warning: PlatformIO expects a 4 MB ESP32 Dev Module while the simulated target reports 2 MB. The firmware nevertheless builds successfully and uses about 20 percent of the configured application flash region. This warning should be resolved in a future configuration cleanup by making the project flash setting match the simulated/physical target.

Other limitations include the use of a relative LDR percentage rather than calibrated lux, the laboratory-scale 15-second inactivity timeout, simulation timing that may differ from physical sensors, and limited handling of DHT22 read failures. On physical hardware, electrical noise, sensor tolerances, pull-up requirements, ADC calibration, switch bounce, I2C wiring, power stability, and real buzzer characteristics would need additional validation.

8. Conclusion

The laboratory demonstrated that a responsive embedded application can be decomposed into cooperating FreeRTOS tasks with clear ownership and communication rules. The final system uses five meaningful tasks, explicit priorities, queues, an event group, a direct task notification, a mutex, periodic scheduling, and an ACTIVE/INACTIVE state machine.

Testing was not limited to showing that the simulator runs. Fourteen native unit tests verified hardware-independent logic, ten Wokwi functional tests exercised integrated behavior, and three deliberate fault experiments demonstrated the effects of removing blocking, changing priority, and removing mutex protection. Static-analysis findings were interpreted and one actionable issue was corrected.

Future improvements should include matching the flash configuration to the target, improving sensor-failure handling, calibrating the light measurement, validating the design on physical hardware, and extending the alarm/state behavior only when supported by clear requirements. The most important lesson is that FreeRTOS mechanisms should be selected for a real engineering purpose and should be explainable in terms of task state, data ownership, synchronization, and timing.

Appendix A. Requirements Traceability Matrix

The matrix connects major functional/RTOS requirements to the implemented component and to verification evidence.

Requirement	Implementation	Verification
FR-01 Environmental measurements	SensorTask reads DHT22/LDR every 2 s	FT-01, FT-02, FT-03
FR-05 Display sensor information	DisplayTask owns SSD1306; sensor_queue	FT-01 to FT-03
FR-06 User navigation	InputTask + rotary encoder + task notification	Unit navigation tests; FT-04, FT-05
FR-07 Temperature alarm	AlarmTask + evaluateTemperature() + buzzer	Alarm unit tests; FT-06, FT-07
FR-09 Motion/state behavior	MotionTask + ACTIVE/INACTIVE logic + event_group	State unit tests; FT-08, FT-09, FT-10
RTOS queue requirement	sensor_queue and alarm_queue	Integrated Wokwi data flow
RTOS mutex requirement	serialMutex protects runtime serial output	Fault Experiment 3
RTOS event/notification requirement	Event group + encoder task notification	FT-08 to FT-10; encoder tests
Periodic scheduling requirement	SensorTask vTaskDelayUntil()	Observed 2 s sampling + design review

Appendix B. Technical Defense Questions and Answers

The following questions come from the laboratory manual's Individual Technical Checkoff. The answers are based on the implemented project.

1. Why did you create SensorTask?

SensorTask gives sensor acquisition one clear responsibility and a predictable two-second schedule. It reads the DHT22 and LDR, packages the values in SensorData, and sends copies to the display and alarm queues instead of mixing measurement logic with the user interface or alarm logic.

2. Why does each task have its assigned priority?

Priorities reflect scheduling urgency. InputTask and MotionTask are priority 3 because they react to external events. SensorTask and AlarmTask are priority 2 because small delays are acceptable. DisplayTask is priority 1 because OLED refresh is least time-critical.

3. What does vTaskDelayUntil() do?

It blocks a periodic task until a calculated absolute wake time based on the previous wake time. This helps SensorTask maintain a regular two-second period and reduces timing drift compared with repeatedly delaying from the current time.

4. What happens to a task while it is delayed?

It enters the Blocked state and does not consume CPU time. The scheduler can run another Ready task until the delay expires.

5. What information crosses your queue?

A SensorData structure containing temperature, humidity, relative light level, and motion status is sent. SensorTask sends a copy to sensor_queue for DisplayTask and alarm_queue for AlarmTask.

6. Why use a queue rather than unsynchronized global variables?

A queue transfers complete data items using FreeRTOS synchronization. It avoids consumers reading a shared structure while another task is updating it and also provides natural blocking when no new data is available.

7. What resource does your mutex protect?

serialMutex protects runtime serial/printf output shared by multiple tasks, especially multi-line diagnostic sections.

8. Where could a race condition occur?

Without synchronization, multiple tasks could write serial output simultaneously, or consumers could read shared sensor/state data while another task changes it. Queues, the event group, notification, and mutex reduce these risks.

9. What does your event group or notification represent?

The event group carries EVENT_ACTIVE, EVENT_MOTION, and EVENT_ALARM state/event bits. The direct task notification is a lightweight wake-up signal from the rotary-encoder GPIO ISR to InputTask.

10. Which task owns the OLED, and why?

DisplayTask is the only OLED owner. Single ownership prevents concurrent I2C/display operations and simplifies synchronization.

11. What happens if a high-priority task never blocks?

It can remain Running whenever it is Ready and can starve lower-priority tasks. The deliberate removal of MotionTask's delay caused delayed behavior and a Task Watchdog event, demonstrating this problem.

12. What is the difference between Ready and Blocked?

Ready means the task is able to run but may be waiting for CPU scheduling. Blocked means it cannot run until an event, queue item, notification, or time delay condition is satisfied.

13. What functionality did your unit tests actually verify?

They verified five alarm threshold/boundary cases, five display-navigation transitions including wrap-around, and four ACTIVE/INACTIVE state-machine cases, for 14 tests total.

14. What did static analysis discover?

The final run reported 0 high, 0 medium, and 7 low unusedFunction style findings. An earlier eighth low finding identified an array that should be const; that issue was corrected. The remaining findings were reviewed in the context of ESP-IDF, C/C++ calls, and native tests.

15. What would differ on physical hardware?

Physical hardware adds sensor tolerance, noise, ADC calibration, switch bounce, wiring and pull-up requirements, I2C electrical behavior, power stability, real timing variation, and actuator characteristics. These would require hardware validation and possibly filtering/debouncing/calibration.